Bismillah ir-Rahman ir-Rahim

FATIMA CORPUS STRENGTHENING PLAN

Formalization of the Bayyinah Research Corpus into Code Structure

Version 1.0 – October 8, 2026

This document maps every actionable pattern extracted from the 995-record Bayyinah Research Corpus (compiled 2026-10-07) and the standalone Mughlaq Trap to concrete code changes across the 33 modules of fatima-core (5,009 LOC, 34/34 tests passing at commit 0af94b0). The plan closes the four remaining Canon findings (F-02, F-06, F-07, F-08), proposes nine adversarial grafts (R-1 through R-9), and introduces eight corpus-derived strengthening grafts that have no existing Canon entry but are demanded by the corpus's own engineering discipline.

READING CONTRACT

1. Every prescription is tagged with its originating corpus record(s).
2. Grafts follow the v0.2 protocol established by GRAFT-F03-PURITY-001: declared replacement, begins at UNKNOWN, rises to VERIFIED only on independent attestation.
3. The ordering is not arbitrary – it follows the dependency graph: infrastructure grafts first, then the grafts that depend on them.
4. Each graft names the file(s) it touches, the lines it replaces, and the test discipline it demands.

PART I: CLOSE THE FOUR REMAINING CANON FINDINGS

GRAFT-F02-SUBSTRATE-001: Syntactic verification acquires an external reference

Finding: F-02 – syntactic verification is circular. `Atom.verify_syntactic()` compares the stored `content_hash` to a recomputed hash of the stored `content`. Both are inside the same untrusted file. A forger who alters `content` need only update `content_hash` to pass L1.

Corpus grounding: - R0097 (Structural Honesty Axioms) – A2 Surface-Substrate Consistency: the surface (stored hash) and the substrate (content) must agree, but agreement between two co-located values that share a trust boundary is not verification. - R0128 (SHIELD) – EC-3 Observational Purity: the verifier must not read from the same store the producer wrote to. - R0151 (Swarm Analysis) – Layer 3 Substrate Discovery: verify what the system *does*, not what it *says* it does.

Prescription: 1. Add an `external_content_hashes` parameter to `verify_syntactic()` that accepts a `dict[str, str]` mapping `atom_id -> expected_hash` from an external source (the provenance envelope, a separate manifest file, or the holographic reconstruction). 2. When `external_content_hashes` is provided: compare each atom's recomputed hash against the external expectation. Mismatch → VIOLATED. 3. When `external_content_hashes` is absent: the current self-check remains but the verdict is capped at UNKNOWN (honest default – cannot prove external consistency without external evidence). 4. Wire the holographic reconstruction's bond-graph as a secondary external reference: after L3 recovers the bond graph, extract the atom content hashes embedded in the shard metadata and feed them to L1 as the

external reference. This makes L1 depend on L3's evidence – a structural cross-reference that no single-atom forgery can satisfy.

Files touched: `fatima/verification/syntactic.py` [23 → ~60], `fatima/core/atom.py` (no change – hash computation is already pure), `fatima/verification/semantic.py` (wire external hashes through L5 pipeline).

Tests required: - Forge a .fatima file with altered content and updated content_hash: L1 without external reference → UNKNOWN (capped). L1 with external reference → VIOLATED. - Round-trip test: encode → save → reload → verify with holographic cross-reference → VERIFIED at L1.

GRAFT-F06-SEMANTIC-001: Bond inference becomes structural, not lexical

Finding: F-06 – bond inference is lexical. `_classify_content()` uses keyword matching (`because`, `must`, `should`). A paraphrase that avoids these keywords changes the molecule's structure. A keyword-loaded inversion can keep its shape while reversing its meaning.

Corpus grounding: - R0097 (Structural Honesty Axioms) – A3 Caller-Side Exhaustiveness: every input state must map to a defined output. The current classifier has a `proposition` catch-all that swallows everything it cannot recognise – the exhaustiveness is achieved by defaulting, not by classification. - R0044 (The Differentiator v6.1) – Layer 6 Semantic Analysis: meaning cannot be inferred from surface tokens alone; the structural position of a claim within its argument matters. - R0101 (Furqan) – T-Zahir-Proj / T-Batin-Proj: every term has a zahir (surface) and batin (substrate) reading. A classifier that sees only zahir (keywords) misses batin (structural role). - R0103 (Munafiq Protocol) – M3 Concealed Intent Tracking: surface compliance with keyword patterns while the structural role differs is exactly the Munafiq failure mode.

Prescription: 1. Replace the keyword-list `_classify_content()` with a two-pass classifier: - **Pass 1 (structural):** classify by document-position rules: first paragraph after a heading → elaboration of heading; paragraph that follows evidence → claim (the thing being supported); paragraph containing a citation → reference. This is structural, not lexical. - **Pass 2 (confirmation):** the keyword heuristics remain but serve as a *confirmation* signal, not the primary classifier. When structural and lexical classifications disagree, mark the atom's `content_type` with a `classification_confidence` field (new) and reduce bond weights for bonds involving low-confidence atoms. 2. Add a `ClassificationReport` dataclass to `document.py` that records the structural classification, the lexical classification, whether they agreed, and the resulting confidence. 3. Wire `classification_confidence` into `verify_fitrah_alignment()` as a new check: a molecule where > 30% of atoms have disagreeing structural/lexical classifications is flagged as UNKNOWN (possible semantic instability). 4. Add adversarial test pairs (from R0103 Munafiq Protocol): - A paragraph that is lexically "evidence" (contains "because") but structurally a claim (first paragraph after a heading with no preceding proposition). - A paragraph that is lexically "claim" (contains "must") but structurally evidence (follows a heading-defined proposition and provides reasoning).

Files touched: `fatima/formats/document.py` [547 → ~620], `fatima/core/atom.py` (add `classification_confidence: float` field), `fatima/verification/semantic.py` (add classification-agreement check).

Tests required: - Paraphrase stability: encode a document, paraphrase every paragraph (remove all keywords), re-encode → the bond graph structure should be 80%+ preserved (measured by Jaccard similarity of bond sets). - Adversarial inversion: encode a document with keywords that contradict structural position → classifier reports disagreement; fitrah check flags the atom.

GRAFT-F07-VECTORISE-001: Holographic encoding becomes vectorised

Finding: F-07 – encoder is slow. `encode_holographic()` runs a Python-level `for byte_val in data:` loop over every byte of the compressed bond graph, creating a fresh polynomial and evaluating it at n points per iteration. At 122 atoms with ~1.8 KB compressed bond graph, this takes 3.84 seconds (measured).

Corpus grounding: - R0487-R0540 (Bayyinah Scanner source) – the scanner's APS scoring loop processes thousands of findings in < 1 second by vectorising the severity × confidence accumulation. The principle: any inner loop over field elements should be a NumPy operation, not a Python loop. - R0203 (Five-Conjunct Kernel) – the 3^5 = 243 exhaustive kernel evaluation demonstrates that brute-force enumeration over small fields is tractable when vectorised.

Prescription: 1. Replace the `for byte_val in data:` loop with a NumPy batch: - Generate all random coefficients at once: `coeffs = np.random.randint(1, 256, size=(len(data), degree), dtype=np.uint8)` - Set the constant terms: `coeffs[:, 0] = data_array` - Actually: restructure as matrix multiplication in GF(2^8). The Vandermonde matrix V[i,j] = x_i^j (where x_i is atom i's evaluation point) is fixed for a given (n, k). Precompute it once. Then shards = V @ coeffs.T in GF(2^8), using `_GF_MUL_TABLE` for the dot product. 2. The decoder is already vectorised (`_vectorised_lagrange_decode`). The encoder must match. 3. Target: < 0.1 seconds for 250 atoms with 20 KB compressed bond graph (the GF(2^8) field maximum).

Files touched: `fatima/core/encoding.py` [528 → ~560].

Tests required: - Round-trip correctness: encode_vectorised → decode → compare. Must produce identical shards to the current scalar path. - Performance benchmark: time the new path at 50, 100, 200, 250 atoms. Assert < 0.5 s at 250 atoms. - Property: the scalar `_make_polynomial` + `_eval_polynomial` path remains as a reference implementation for testing. The vectorised path must agree byte-for-byte.

GRAFT-F08-GRACEFUL-001: Shard corruption yields a localised verdict

Finding: F-08 – shard corruption fails by exception. `verify_holographic_reconstruction()` catches zlib decompression errors and returns `(False, f"Reconstruction failed: {e}")`. This conflates corruption (which should be VIOLATED with localisation) with structural issues (which might be UNKNOWN).

Corpus grounding: - R0487-R0540 (Scanner source) – error-to-finding conversion: errors become findings with `integrity_score` clamped to max 0.50, not exceptions that abort the pipeline. - R0152 (Heartbeat of a Live System) – defect migration: a defect must be classified and tracked, never silently absorbed. - R0097 (Structural Honesty Axioms) – A1 Append-Only Provenance: the record of failure is itself evidence that must be preserved. - R0128 (SHIELD) – BSSG three-valued conjunction: the conjunction of findings must compose through the three-valued lattice, never through exception handling.

Prescription: 1. Replace the bare `except Exception` in `verify_holographic_reconstruction` with structured error classification: - `zlib.error` → VIOLATED: "Shard data corrupted – decompression failed at byte offset {e.args}" with the specific shard IDs that contributed to the failed reconstruction. - `json.JSONDecodeError` → VIOLATED: "Bond graph reconstruction produced invalid JSON – structural corruption beyond decompression" with the position of the JSON error. - `KeyError` / `ValueError` in bond deserialization → VIOLATED: "Reconstructed bond graph has invalid structure" with the specific bond or field that failed. - Any other exception → UNKNOWN: "Reconstruction failed for unexpected reason" with the exception

type and message (never swallow the exception silently). 2. Add per-shard MAC (message authentication code) to detect which specific shard(s) are corrupted before attempting reconstruction: - During `encode_holographic()`: compute HMAC-SHA256 of each shard using the bond_graph_hash as the key. Store the MAC alongside the shard in the atom. - During `verify_holographic_reconstruction()`: check each shard's MAC before attempting Lagrange interpolation. A shard with a failed MAC is excluded from the reconstruction set. If fewer than k valid shards remain → VIOLATED with "Insufficient uncorrupted shards: {valid}/{total}, need {k}". - This localises corruption to specific atoms, enabling partial recovery with the remaining valid shards. 3. Return a `VerificationReport` instead of `tuple[bool, str]` to integrate with the three-valued verdict lattice.

Files touched: `fatima/core/encoding.py` [528 → ~600], `fatima/core/atom.py` (add `shard_mac: str` field), `fatima/verification/holographic.py` (consume the new MAC field).

Tests required: - Corrupt one shard → specific shard identified, reconstruction uses remaining shards, verdict is VERIFIED (if k-1 >= threshold). - Corrupt enough shards that < k remain → VIOLATED with localised report naming the corrupted atoms. - Corrupt the decompressed data (inject bad JSON) → VIOLATED with JSON-specific error message. - Round-trip with MACs: encode → verify MACs → decode → compare.

PART II: CORPUS-DERIVED STRENGTHENING GRAFTS

These grafts have no existing Canon entry. They are demanded by patterns observed across the corpus that the current codebase does not implement.

GRAFT-C01-PRODUCER-QUARANTINE: No component certifies its own output

Corpus grounding: - R0097 (Structural Honesty Axioms) – D6 producer-quarantine: the producer of a datum must not be the evaluator of that datum. - R0145 (SHV Eight Disciplines) – D6 is one of the eight disciplines. - R0146 (SHV A New Frontier) – P3 recursive self-application: the system must verify its own verification logic, but the verifier of the verification must be distinct from the original verifier.

Current violation: `document.py:encode_document()` calls `apply_holographic_encoding()` and then `compute_all_semantic_hashes()` in the same pipeline. The encoded molecule is the producer's output. The semantic hashes are computed by the producer. No quarantine boundary exists between production and commitment.

Prescription: 1. Split `encode_document()` into two phases: - `encode_document()` → returns `MoleculeEnvelope(molecule, encoding_params)` *without* computing semantic hashes or bond-graph hash. - `finalise_document(envelope)` → computes and commits the hashes. This function lives in a separate module (`fatima/formats/finalise.py`) that shares no mutable state with `document.py`. 2. The CLI `encode` command calls both in sequence, but the separation makes the quarantine boundary visible and testable. 3. Add a `produced_by` field to `Molecule` metadata that records which code path produced it. `finalise_document` refuses to finalise a molecule whose `produced_by` matches its own module path.

Files touched: New file `fatima/formats/finalise.py` [~80 LOC], `fatima/formats/document.py` (remove hash computation), `fatima/cli.py` (call both phases).

GRAFT-C02-FAIL-CLOSED: Missing data pushes AWAY from PASS

Corpus grounding: - R0203 (Five-Conjunct Kernel) – fail-closed imputation: in the 3^5 = 243 exhaustive kernel, any UNKNOWN conjunct produces UNKNOWN or FAIL, never PASS. - R0128 (SHIELD) – BSSG conjunction: TRUE ∧ UNKNOWN = UNKNOWN. - R0097 (Structural Honesty Axioms) – NC-P4: UNKNOWN cannot authorise irreversible effect.

Current gap: `verify_fitrah_alignment()` returns `Verdict.UNKNOWN` when violations are found, but the confidence calculation `1.0 - (len(violations) / (total_checks * 2))` can still produce high confidence (0.5+) with multiple violations. This is not fail-closed – the score drifts toward the centre rather than toward zero.

Prescription: 1. Revise the confidence formula in `verify_fitrah_alignment()`: - confidence = `max(0.0, 1.0 - (len(violations) / total_checks))` - Each violation reduces confidence by `1/total_checks`, not by `1/(2 * total_checks)`. A finding that halves its impact is not a finding; it is a suggestion. 2. Apply the same discipline to `check_meaning_integrity()`: - When UNKNOWN (missing evidence), confidence should be `examined_with_commitments / total_atoms`, not the current formula that mixes violations and unknowns. 3. Add a `scan_incomplete` boolean to `VerificationReport`: - True when any check could not be performed (missing commitments, absent shards, no provenance). - When `scan_incomplete` is true, the confidence is clamped to max 0.50 (the Scanner kernel's discipline from R0487).

Files touched: `fatima/verification/verdict.py` (add `scan_incomplete`), `fatima/verification/semant` `fatima/properties/meaning.py`, `fatima/verification/holographic.py`.

GRAFT-C03-KILL-SWITCH: Non-waivable abort conditions

Corpus grounding: - R0097 (Structural Honesty Axioms) – the four axioms are non-waivable: violating any one forces the verdict to VIOLATED regardless of how many other checks pass. - R0102/R0153 (Mughlaq Trap) – Tier 0 verdict floor: a Tier 0 finding forces the system verdict to mughlaq regardless of downstream findings. - R0487 (Scanner source) – kill-switch patterns: certain finding codes (e.g., format-mismatch between extension and magic bytes) force `scan_incomplete = True` and clamp the score to ≤ 0.50.

Current gap: The verdict lattice composes by meet (worst of two), which is correct. But there is no mechanism to declare a finding as *non-waivable* – a finding so severe that it cannot be outweighed by passing checks. The lattice meet handles this for VIOLATED (VIOLATED ∧ anything = VIOLATED), but the system has no way to *force* VIOLATED from a single check when the check would otherwise produce UNKNOWN.

Prescription: 1. Add a `KillSwitch` mechanism to `VerificationReport`: - A new field `kill: bool = False`. - When `kill` is True, `VerificationReport.compose()` immediately returns VIOLATED regardless of other reports. - Kill conditions: - Self-attestation detected (reviewer_key == author_key in L5). - Bond graph hash mismatch with external reference present (L1+L3). - Shard MAC failure on > 50% of atoms (L3). 2. Wire the Mughlaq Trap's verdict-floor concept: if any single check returns `kill=True`, the composed verdict is VIOLATED with the kill finding listed first in the violations tuple.

Files touched: `fatima/verification/verdict.py` (add `kill` field), `fatima/verification/provenance.py` (set kill on self-attestation), `fatima/verification/holographic.py` (set kill on majority MAC failure).

GRAFT-C04-ERROR-TO-FINDING: Errors become findings, not exceptions

Corpus grounding: - R0487-R0540 (Scanner source) – the scanner converts every exception into a finding with severity and confidence, then clamps the integrity score. No exception aborts the pipeline; every exception is evidence. - R0097 (Structural Honesty Axioms) – A1 Append-Only Provenance: the record of failure is evidence. Swallowing an exception destroys evidence.

Current gap: Multiple modules use bare `try/except` blocks that convert exceptions to boolean failure or string messages without structured tracking: - `encoding.py:verify_holographic_recons` → `(False, str)` - `provenance.py:verify_provenance` → catches `Exception` silently - `holographic.py:verify_holographic` → catches reconstruction errors

Prescription: 1. Define a `Finding` dataclass in a new module `fatima/verification/finding.py`: `@dataclass(frozen=True) class Finding: code: str # e.g., "F-HOLO-DECOMP-001" severity: float # 0.0-1.0 confidence: float # 0.0-1.0 surface: str # zahir -- what the finding looks like concealed: str # batin -- what it means structurally source_module: str # which module detected it kill: bool = False # non-waivable?` 2. Every except block in the verification pipeline converts to a `Finding` with appropriate severity and confidence. 3. `VerificationReport` gains a `findings: tuple[Finding, ...]` field alongside the existing `violations: tuple[str, ...]` (which becomes a derived property for backward compatibility). 4. The integrity score is computed as: `score = clamp(1.0 - sum(f.severity * f.confidence for f in findings), 0.0, 1.0)`

Files touched: New file `fatima/verification/finding.py` [~60 LOC], `fatima/verification/verdict.py` all verification modules.

GRAFT-C05-TWO-SIDED: Every check verifies both directions

Corpus grounding: - R0044 (The Differentiator v6.1) – Guardrail 3: security/correctness AND functionality/capability must both be checked. - R0145 (SHV Eight Disciplines) – D4 two-sided verification: a check that only looks for the presence of a property, not for the absence of its negation, is half a check. - R0146 (SHV A New Frontier) – P2 falsifiability: every property must have a defined failure mode. If you cannot describe what failure looks like, you have not described the property.

Current gap: `verify_fitrah_alignment()` checks for the *presence* of structural indicators (claims have support, definitions have elaboration) but does not check for the *absence* of expected structure: - A document with zero claims is not flagged (it should be: a document that makes no claims has no falsifiable content). - A document with zero definitions is not flagged (it should be: a document that defines nothing builds on undefined terms).

Prescription: 1. Add undergeneration checks to `verify_fitrah_alignment()`: - Zero claims in a molecule with > 5 atoms → UNKNOWN: "No falsifiable claims detected (possible undergeneration)" - Zero definitions in a molecule with > 10 atoms → UNKNOWN: "No definitions detected (possible undefined-term risk)" - Zero evidence in a molecule with > 3 claims → UNKNOWN: "Claims without any evidence (possible overgeneration)" 2. Add a `coverage_ratio` field to `VerificationReport`: the fraction of the expected content-type distribution that is present.

Files touched: `fatima/verification/semantic.py` (new checks), `fatima/verification/verdict.py` (add `coverage_ratio`).

GRAFT-C06-EXHAUSTIVE-KERNEL: The 243-row decision matrix

Corpus grounding: - R0203 (Five-Conjunct Kernel) – 3^5 = 243 exhaustive enumeration of the decision kernel. Every combination of {TRUE, FALSE, UNKNOWN} across the five properties must have a defined outcome. - R0128 (SHIELD) – BSSG conjunction: the three-valued conjunction table is small enough to enumerate exhaustively.

Current gap: The verdict composition in `VerificationReport.compose()` takes the worst verdict and minimum confidence, but the system has never been tested against the complete 3^5 = 243 decision matrix. Edge cases (e.g., three VERIFIED + one UNKNOWN + one VIOLATED) may not compose correctly in all orderings.

Prescription: 1. Generate the exhaustive 3^5 = 243 test matrix. 2. For each combination, verify that `compose()` returns the expected verdict: - Any VIOLATED → VIOLATED - All VERIFIED → VERIFIED - Otherwise → UNKNOWN 3. Verify associativity and commutativity of the lattice meet: for all orderings of the five properties, the result must be identical. 4. This is a test-only graft – no production code changes if the lattice is already correct. If the tests reveal edge cases, fix them.

Files touched: New test file `tests/test_exhaustive_kernel.py` [~120 LOC].

GRAFT-C07-BARZAKH-GATE: Three-question filter at every trust boundary

Corpus grounding: - R0044 (The Differentiator v6.1) – the Barzakh Gate: three questions at every trust boundary: (1) By what authority? (2) By what methodology? (3) From what lineage? - R0097 (Structural Honesty Axioms) – A4 Projection Consistency: every projection must be consistent with the source.

Current gap: The provenance envelope checks authority (author key) and partially checks methodology (signature verification), but does not check lineage (the chain of transformations that produced the molecule). A molecule that was produced by a chain of three encode→modify→re-encode operations has no record of the intermediate steps.

Prescription: 1. Add a `lineage: list[LineageStep]` field to `Molecule.metadata`: `@dataclass(frozen=True) class LineageStep: operation: str # "encode", "modify", "re-encode", "graft" timestamp: str # ISO-8601 agent: str # who performed the operation input_hash: str # hash of the input output_hash: str # hash of the output` 2. Every operation that modifies a molecule appends a `LineageStep`. 3. `verify_provenance()` gains a lineage-consistency check: the output hash of step N must equal the input hash of step N+1. Any gap → UNKNOWN: "Lineage gap between steps N and N+1".

Files touched: New file `fatima/core/lineage.py` [~50 LOC], `fatima/core/molecule.py` (store lineage in metadata), `fatima/verification/provenance.py` (lineage check).

GRAFT-C08-RECURSIVE-SELF: The system verifies its own verification

Corpus grounding: - R0146 (SHV A New Frontier) – P3 recursive self-application: the framework must verify its own verification logic. This is the strongest form of structural honesty. - R0152 (Heartbeat of a Live System) – the 25 consecutive Class A rounds demonstrate that a system that monitors itself can maintain integrity over time. - R0092 (FATIMA paper) – "Mode collapse fails whenever the whole is present in every part."

Current gap: The verification pipeline verifies molecules, but it does not verify itself. If `verify_holographic()` contains a bug that always returns VERIFIED, nothing in the system detects this.

Prescription: 1. Create `fatima/verification/self_check.py`: - `verify_verification_integrity()` that: a. Creates a known-bad molecule (one atom with altered content but matching content_hash – the F-02 forgery). b. Runs the full L1-L5 pipeline. c. Asserts the result is NOT VERIFIED. If it is → the verification pipeline itself is compromised. - `verify_lattice_integrity()` that: a. Tests all nine cells of the 3×3 lattice meet table. b. Asserts VIOLATED & VERIFIED = VIOLATED, etc. c. Tests associativity and commutativity. 2. The CLI gains a `fatima self-check` command that runs these and reports the result. 3. The self-check is the first thing `fatima verify` runs (before verifying the user's molecule). If the self-check fails, all verification is UNKNOWN: "Verification pipeline failed self-check."

Files touched: New file `fatima/verification/self_check.py` [~100 LOC], `fatima/cli.py` (add self-check command and pre-verify hook).

PART III: ADVERSARIAL AUDIT REPAIRS (R-1 THROUGH R-9)

These are the nine proposed repairs from the v0.2 adversarial audit (F-V02-01 through F-V02-11). Each repair is grounded in its original audit finding and reinforced by corpus records.

R-1: Paraphrase-immune bond inference

Audit finding: F-V02-01 – bond graph changes under paraphrase. **Maps to:** GRAFT-F06-SEMANTIC-001 (above). **Status:** Subsumed by GRAFT-F06.

R-2: Negation-stable classification

Audit finding: F-V02-02 – negating a paragraph does not change its content_type. **Corpus reinforcement:** R0103 (Munafiq Protocol) M4 – Inversion of Declared Position: the surface form says one thing while the structural role says the opposite.

Prescription: 1. Add a `polarity` field to `Bond`: `positive` or `negative`. A SUPPORTS bond with `polarity=negative` is a CONTRADICTS bond. 2. `_infer_bonds()` detects negation markers ("not", "never", "no", "fails to", "does not") and sets polarity accordingly. 3. Fitrah check: a molecule where > 40% of SUPPORTS bonds have negative polarity is flagged as internally contradictory.

Files touched: `fatima/core/bond.py`, `fatima/formats/document.py`, `fatima/verification/semantic.p`

R-3: Content-type adversarial test suite

Audit finding: F-V02-03 – no adversarial test fixtures exist. **Maps to:** GRAFT-F06 test requirements (above) plus:

Prescription: 1. Expand `fatima/graft/adversarial/fixtures.py` with 20 paired test cases: (original, adversarial_variant, expected_difference). 2. Categories: - Paraphrase pairs (same meaning, different keywords) - Negation pairs (opposite meaning, same structure) - Keyword-loaded inversions (adversarial keywords, reversed meaning) - Structural rearrangements (same content, different document order)

R-4: Holographic reconstruction with partial corruption

Audit finding: F-V02-04 – no test for partial shard corruption. **Maps to:** GRAFT-F08-GRACEFUL-001 (above). **Status:** Subsumed by GRAFT-F08.

R-5: Bond weight sensitivity analysis

Audit finding: F-V02-05 – bond weights are chosen by heuristic with no sensitivity analysis.

Prescription: 1. Add a sensitivity test: for each bond weight in the encoding pipeline, perturb by ±0.1 and verify that: - Fitrah alignment verdict does not change (weights are not decision-critical at the margin). - Holographic reconstruction succeeds (weights do not affect shards). 2. If any weight perturbation flips a verdict → that weight is a critical parameter and must be documented with its rationale.

Files touched: New test file `tests/test_weight_sensitivity.py`.

R-6: Composite molecule cross-boundary forgery

Audit finding: F-V02-06 – inter-molecule bonds are not covered by holographic encoding.

Prescription: 1. Include inter-bond metadata in the composite hash computation. 2. Add a test: alter an inter-bond weight → composite hash mismatch detected → VIOLATED.

Files touched: `fatima/core/composite.py`.

R-7: Serialisation round-trip fidelity

Audit finding: F-V02-07 – no test that save→load→save produces identical bytes.

Prescription: 1. Add a round-trip idempotency test: encode → save → load → save → compare bytes. The two saved files must be identical. 2. If floating-point precision causes drift, pin the JSON serialisation to a fixed decimal precision.

Files touched: New test file `tests/test_roundtrip.py`.

R-8: Verify that verification is not identity

Audit finding: F-V02-08 – no test that verification rejects a random molecule.

Prescription: 1. Generate a random molecule (random content, random bonds, random shards) and run full L1-L5 verification. 2. Assert: the result is NOT VERIFIED. If it is → the verification pipeline accepts anything, which means it verifies nothing. 3. This is the falsifiability test for the verification pipeline itself.

Files touched: New test file `tests/test_falsifiability.py`.

R-9: Provenance envelope completeness

Audit finding: F-V02-09 – the provenance envelope has no required fields beyond `author_key_id` and `signature`.

Prescription: 1. Add a `validate()` method to `ProvenanceEnvelope` that checks: - `commitment_hash` is non-empty. - `anchor_provider` is not "null" (warns if it is). - `author_key_id` is non-empty. - `signature` is non-empty. 2. `verify_provenance()` calls `validate()` first. Invalid envelope → VIOLATED (not UNKNOWN – presenting an invalid envelope is worse than presenting no envelope).

Files touched: `fatima/verification/provenance.py`.

PART IV: IMPLEMENTATION ORDER

The dependency graph determines the order:

```
Phase 1 (Infrastructure):
  C04-ERROR-TO-FINDING     → Finding dataclass
  C02-FAIL-CLOSED          → scan_incomplete, confidence formula
  C03-KILL-SWITCH          → kill field on VerificationReport

Phase 2 (Core Fixes):
  F-08-GRACEFUL-001        → depends on C04 (findings, not exceptions)
  F-07-VECTORISE-001       → independent
  F-02-SUBSTRATE-001       → depends on C02 (fail-closed for capped verdicts)

Phase 3 (Semantic):
  F-06-SEMANTIC-001        → depends on C02 (classification confidence)
  C05-TWO-SIDED            → depends on F-06 (content-type awareness)
  R-2 (negation polarity)  → depends on F-06

Phase 4 (Structural):
  C01-PRODUCER-QUARANTINE  → depends on nothing but is a refactor
  C07-BARZAKH-GATE         → depends on C01 (lineage tracking)
  R-6 (composite forgery)  → independent

Phase 5 (Verification):
  C06-EXHAUSTIVE-KERNEL    → test-only, depends on C02+C03
  C08-RECURSIVE-SELF       → depends on everything above
  R-8 (falsifiability)     → depends on C08
  R-3, R-5, R-7, R-9       → independent tests and fixes
```

Total estimated LOC change: +800 to +1,000 LOC (new modules + expanded tests), bringing fatima-core from 5,009 to approximately 5,800-6,000 LOC.

Test count target: 34 existing + ~30 new = ~64 tests.

PART V: CORPUS RECORD INDEX

Every graft in this plan traces to one or more of these source records:

Record	Title	Graft(s)
R0029	The Most Beautiful Names	Design philosophy
R0044	The Differentiator v6.1	C05, C07, F-06
R0092	FATIMA (paper)	C08, design philosophy
R0097	Structural Honesty Axioms	F-02, C01, C02, C03, C04, C07
R0101	Furqan	F-06 (zahir/batin)
R0102/R0153	The Mughlaq Trap	C03 (verdict floor)
R0103	Munafiq Protocol	F-06, R-2
R0128	SHIELD	F-02, C02, C06
R0140-R0141	SHUDP	Design philosophy
R0145	SHV Eight Disciplines	C01 (D6), C05 (D4)

Record	Title	Graft(s)
R0146	SHV A New Frontier	C01 (P3), C05 (P2), C08 (P3)
R0151	Swarm Analysis	F-02 (Layer 3)
R0152	Heartbeat of a Live System	C04, C08
R0196	XZ Protocol Results	C06 (H=1.0)
R0203	Five-Conjunct Kernel	C02, C06
R0487-R0540	Scanner Source	F-07, F-08, C02, C04

This plan's own verdict about itself is UNKNOWN until the grafts are applied, tested, and independently reviewed. The plan is the map; the territory is the code.

Bismillah ir-Rahman ir-Rahim